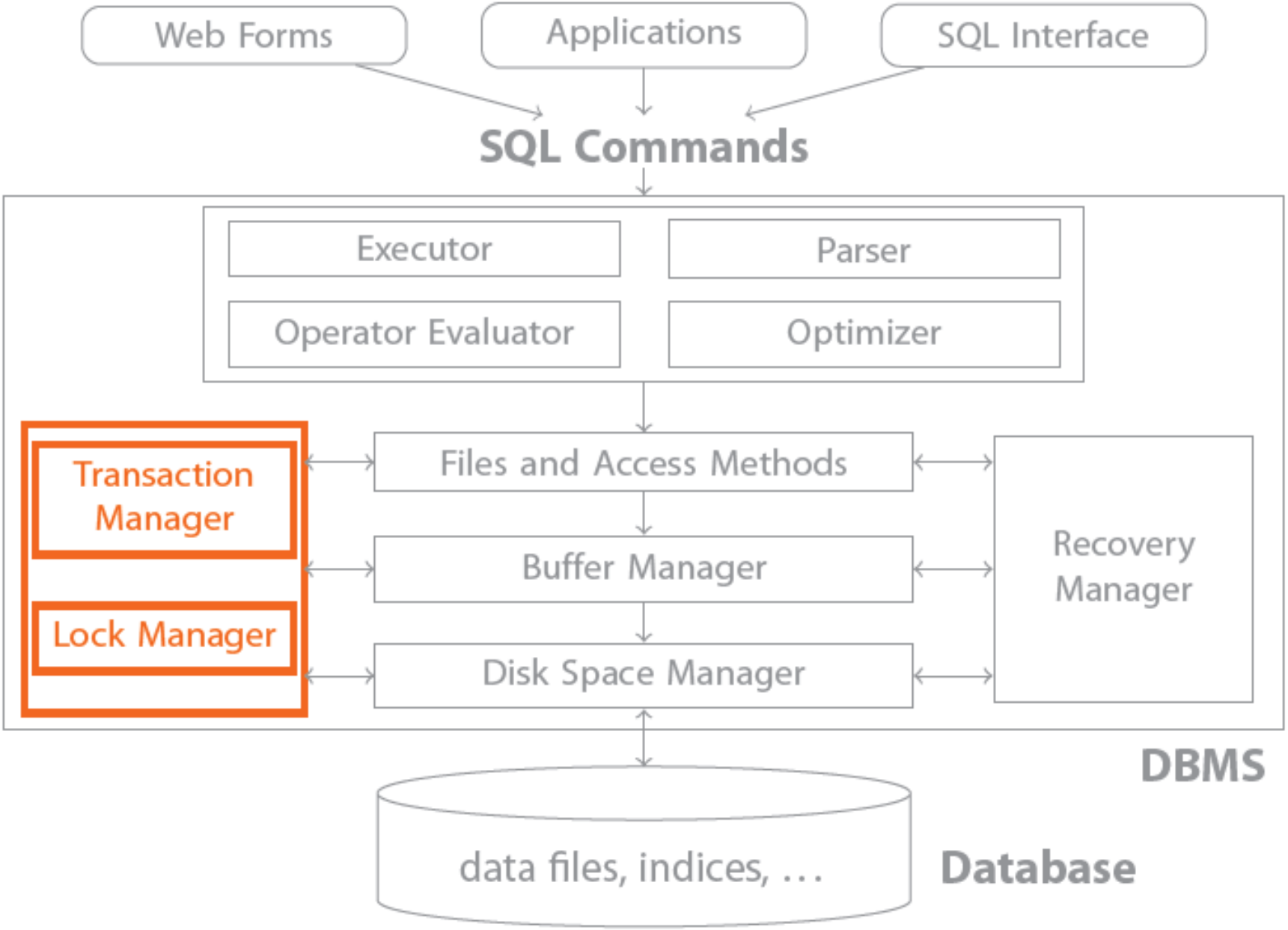
Database Systems Implementation

Transactions and Concurrency

Boris Motik

Slides inspired by Ramakrishnan/Gehrke: ‘Database Management Systems‘, McGraw-Hill 2003;
Dan Olteanu; Amir Shaikhha; Tim Furche; Emanuel Sallinger; Michael Benedikt; Thorsten Grust.
Many figures courtesy Thorsten Grust.


Web Forms
Applications
SQL Interface
SQL Commands
Executor
Parser
Operator Evaluator
Optimizer
Transaction Manager
Lock Manager
Files and Access Methods
Buffer Manager
Disk Space Manager
Recovery Manager
DBMS
data files, indices, …
Database

The ‘Hello World’ of Transaction Management

- I use my debit card at an ATM to draw some money from my account, causing the ATM to perform a **transaction** in the bank’s database.

Example (ATM transaction)

$bal \leftarrow \texttt{read_bal}\,(acct_no)\;;$
$bal \leftarrow bal - 100\,€\;;$
$\texttt{write_bal}\,(acct_no, bal)\;;$

- Expectation: my account is **properly updated** to reflect the new balance.

Concurrent Access: A Family Affair

- Problem: my partner has a card for the same account.
- We might end up using our cards at different ATMs at the same time—that is, concurrently.

me	my wife	DB state
$bal \leftarrow \texttt{read}\,(acct)\,;$		1200
	$bal \leftarrow \texttt{read}\,(acct)\,;$	1200
$bal \leftarrow bal - 100\,;$		1200
	$bal \leftarrow bal - 200\,;$	1200
$\texttt{write}\,(acct\,,\,bal)\,;$		1100
	$\texttt{write}\,(acct\,,\,bal)\,;$	1000

- The first update was **lost** during execution. Lucky me! 😀

If the Plug is Pulled…

- I want to transfer money over to another account.

```
  // Subtract money from source (checking) account
1 chk_bal ← read_bal (chk_acct_no) ;
2 chk_bal ← chk_bal − 500 € ;
3 write_bal (chk_acct_no, chk_bal) ;

  // Credit money to the target (savings) account
4 sav_bal ← read_bal (sav_acct_no) ;
5 sav_bal ← sav_bal + 500 € ;
6   ↯
7 write_bal (sav_acct_no, sav_bal) ;
```

- Before the transaction gets to step 7, its execution is interrupted/cancelled (power outage, disk failure, software bug,…). My money is **lost**! 😱

ACID Properties

To prevent such effects from happening, a DBMS guarantees the following transaction properties:

- **A**tomicity: Either all or none of the updates in a database transaction are applied.
- **C**onsistency: Each transaction brings the database from one consistent state to another. (Database state may be temporarily inconsistent while a transaction runs.)
- **I**solation: A transaction must not see any effect from other transactions that run in parallel.
- **D**urability: The effects of a successful transaction remain persistent and cannot be undone for system reasons.

Lost Update Anomaly

- The effects of one transaction are lost due to an uncontrolled overwrite by the second transaction.
  - Recall example with my partner's card:

me	my wife	DB state
$bal \leftarrow \texttt{read}(acct)$;		1200
	$bal \leftarrow \texttt{read}(acct)$;	1200
$bal \leftarrow bal - 100$;		1200
	$bal \leftarrow bal - 200$;	1200
$\texttt{write}(acct, bal)$;		1100
	$\texttt{write}(acct, bal)$;	1000

Inconsistent Read Anomaly

Transaction 1

```
1 UPDATE Accounts
2    SET balance = balance - 500
3  WHERE customer = 1904
4    AND account_type = 'C';

5 UPDATE Accounts
6    SET balance = balance + 500
7  WHERE customer = 1904
8    AND account_type = 'S';
```

Transaction 2

```
1 SELECT SUM(balance)
2   FROM Accounts
3  WHERE customer = 1904;
```

- Transaction 2 sees a temporary, inconsistent database state.

Dirty Read Anomaly

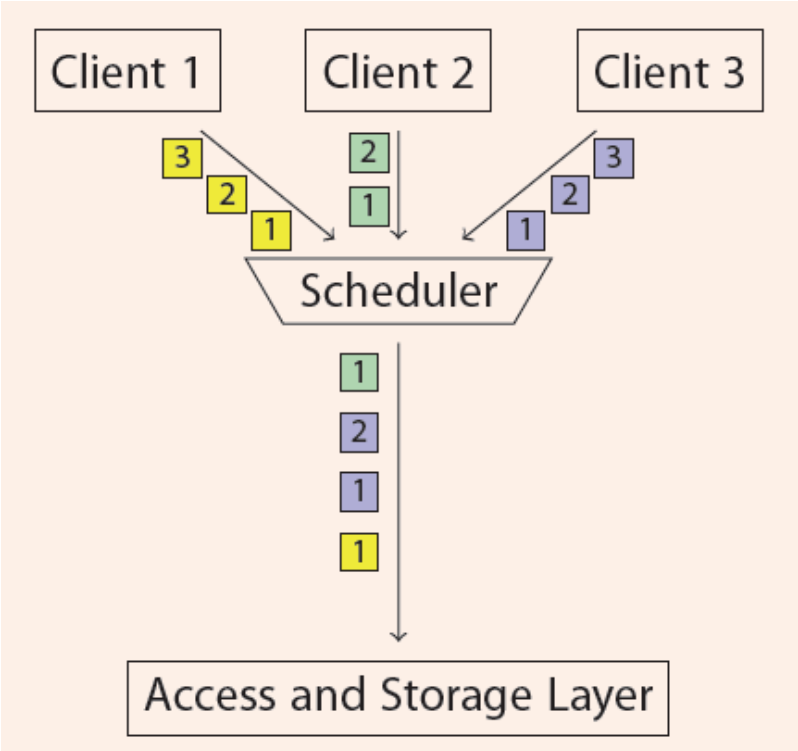
- My partner and I again use an ATMs at roughly the same time. This time, my transaction is cancelled (**aborted**).

me	my wife	DB state
$bal \leftarrow \texttt{read}(acct)$;		1200
$bal \leftarrow bal - 100$;		1200
$\texttt{write}(acct, bal)$;		1100
	$bal \leftarrow \texttt{read}(acct)$;	1100
	$bal \leftarrow bal - 200$;	1100
$\texttt{abort}$;		1200
	$\texttt{write}(acct, bal)$;	900

- My partner's transaction has already read the modified account balance before my transaction was **rolled back** (i.e., its effects are undone).

Concurrent Execution

- The transaction **scheduler** decides the execution order of concurrent database accesses.



What is a Correct Execution?

- Anomalies such as the ‘lost update’ problem can **only** occur with multiple transactions.
- If all transactions were fully executed one after another (no concurrency), no anomalies would occur.
- **Any execution where one transaction is evaluated before another is thus correct!**
  - = serial execution

- Correctness criterion:
  Allow concurrent execution if the **overall effect** is the same as some (arbitrary) serial schedule.

Independence of Computation

T_1	T_2	A	B
		25	25
t := READ(A)			
t := t + 100			
WRITE(A,t)		125	
	s := READ(A)		
	s := s * 1		
	WRITE(A,s)	125	
	s := READ(B)		
	s := s * 1		
	WRITE(B,s)		25
t := READ(B)			
t := t + 100			
WRITE(B,t)			125

Operations of T_2 are idempotent, so we can move them to the front or back ⇒ computation is serializable.

Independence of Computation

T_1	T_2	A	B
		?	?
READ(A)			
WRITE(A)		?	
	READ(A)		
	WRITE(A)	?	
	READ(B)		
	WRITE(B)		?
READ(B)			
WRITE(B)			?

Schedulers ensure serializability by looking just at read/write operations on database elements and assume that any value can be read/written.

Database Elements and Accesses

Assume a slightly simplified model of database access:

- A database consists of a number of named **elements**. In a given database state, each element has a **value**.
- Transactions access an element **e** using the two operations **read e** and **write e**.

Possibilities in relational DBMSs:

- element = attribute
- element = row
- element = table

Database Transactions

- A database transaction T is a sequence of steps, each applying an action to a database element.
  - Transaction T is a sequence of **steps**
  - Step: $r_T(X)$ or $w_T(X)$
    - Action: r (read) or w (write)
    - X is a database element (e.g., table, row, column)
    - Subscript T says that the step belongs to transaction T
- The **length** |T| of a transaction T is the # of its steps.
- Example (the money transfer transaction):
  - T: $read_T(Chk)$, $write_T(Chk)$, $read_T(Sav)$, $write_T(Sav)$
- We abbreviate steps of transaction T_i as $r_i(X)$ and $w_i(X)$.
  - instead of $r_{Ti}(X)$ and $w_{Ti}(X)$

Schedules

- A **schedule** S for a set of transactions { T_1,…, T_n } is a sequence of all steps of the given transactions that preserves the order of steps of each transaction.
  - If p_i precedes q_i in transaction T_i, then p_i precedes q_i in S.

- A schedule specifies how the steps of several transactions interleave on the database.

Serial Schedules

- A schedule is **serial** if all steps of each transaction are adjacent.
  - No interleaving of transactions and thus no concurrency

Examples:

- Serial: $r_1(B), w_1(B), r_2(B), w_2(B)$
- Not serial: $r_1(B), r_2(B), w_1(B), w_2(B)$

Correctness of Serial Schedules

- Anomalies such as the ‘lost update’ problem can **only** occur with multiple transactions.
- If all transactions were fully executed one after another (no concurrency), no anomalies would occur.
- **Any serial schedule is thus correct, BUT…**
- …disallowing concurrent schedules is **impractical**.

- Correctness criterion:

  Allow concurrent schedules if their **overall effect** is the same as some (arbitrary) serial schedule.

Conflict-Equivalence

- When are two schedules S and S' equivalent?
- We can swap adjacent steps $p_i(X)$ and $q_j(Y)$ in a schedule without affecting the outcome if
  - the steps belong to different transactions (i.e., $T_i \neq T_j$), and
  - the steps do not **conflict**—that is, it is not the case that
    - X = Y and
    - either p_i or q_i (or both) are write operations.
- S and S' are **conflict-equivalent** if S can be transformed to S' by repeated swaps as above.

Conflict Serializability

- A schedule S is **conflict-serializable** if it is conflict-equivalent to some serial schedule S'.
  - The execution of a conflict-serializable schedule S is correct.
  - S does not have to be a serial schedule.

Example

T_1	T_2
r(A)	
w(A)	
	r(A)
	w(A)
r(B)	
w(B)	
	r(B)
	w(B)

Swap r(B) and w(B) of T_1 with r(A) and w(A) of T_2: allowed because database elements differ.

Serial schedule!

T_1	T_2
r(A)	
w(A)	
r(B)	
w(B)	
	r(A)
	w(A)
	r(B)
	w(B)

Example

T_1	T_2
r(A)	
w(A)	
	r(A)
	w(A)
	r(B)
	w(B)
r(B)	
w(B)	

Cannot swap r(A) of T_1 with w(A) of T_2, or r(B) of T_1 with w(B) of T_2: not serializable.

Conflict (Serialization) Graph

Graph G(S):

- There is one node for each transaction T_i in S.
- There is an edge from T_i to T_j if and only if S contains actions $p_i(X)$ and $q_j(Y)$ such that
  - $p_i(X)$ comes before $q_j(Y)$ in S,
  - X = Y, and
  - either p_i or q_j (or both) is a write operation.

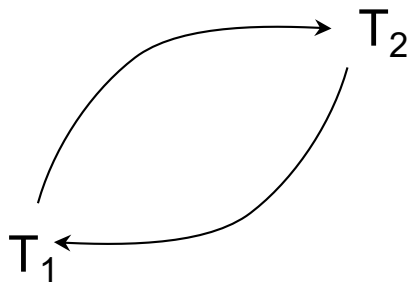
S is conflict-serializable if and only if G(S) is acyclic.

An equivalent serial schedule for S may be obtained by sorting G(S) topologically.

Example

- Our example with ATM transactions:

T_1	T_2
r(A)	
	r(A)
w(A)	
	w(A)

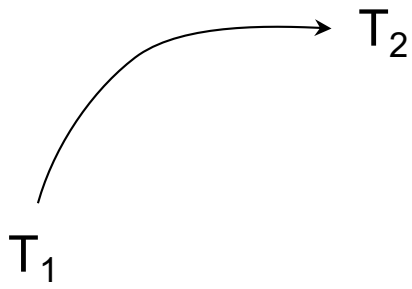


Cycle: not serializable!

Example

- Another example:

T_1	T_2
r(A)	
w(A)	
	r(A)
	w(A)
r(B)	
w(B)	
	r(B)
	w(B)



Serializable!

Conflict-Serializability is NOT Necessary for Serializability

- Transactions T_1, T_2, T_3 write values for X after writing values for Y
- S_1: $w_1(Y)$, $w_1(X)$, $w_2(Y)$, $w_2(X)$, $w_3(X)$
  - Serial: leaves X with the value written by T_3 and Y with the value written by T_2
- S_2: $w_1(Y)$, $w_2(Y)$, $w_2(X)$, $w_1(X)$, $w_3(X)$
  - Not conflict-serializable!
  - Outcome after S_2 is the same as for S_1, so S_2 is serializable!
  - The values written by T_1 and T_2 have no effect and are overwritten later.

Query Scheduling

Can we build a scheduler that **always** emits a serializable schedule?

Idea:

- Require each transaction to obtain a **lock** before it accesses a data element **e**

  lock e;

  … access e …;

  unlock e;

- This prevents **concurrent** access to **e**.

Locking

- If a lock cannot be granted (e.g., because another transaction T' already holds a **conflicting** lock) the requesting transaction T gets **blocked**.
- The scheduler **suspends** execution of the blocked transaction T.
- Once T' **releases** its lock, it may be granted to T, whose execution is then **resumed**.

Since other transactions can continue execution while T is blocked, locks can be used to **control the relative order of operations**.

Lock Modes

- Two **read** operations do not conflict with each other.
- Systems typically use different types of locks (**lock modes**) to allow read operations to run concurrently.
  - **read locks** or **shared locks**: mode S
  - **write locks** or **exclusive locks**: mode X
- Conflict: if at least one lock is an X lock
- It is a safe operation in two-phase locking to (try to) **convert a shared lock into an exclusive lock during the lock phase** (lock upgrade)
  - Effect: improved concurrency

Locking and Scheduling

- Consider two transactions $T_{1,2}$:

T_1	T_2
lock A	lock A
write A	write A
lock B	lock B
unlock A	write B
write B	write A
unlock B	unlock A
	write B
	unlock B

- Two valid schedules (respecting `lock` and `unlock` calls) are:

Schedule S_1

T_1	T_2
lock A	
write A	
lock B	
unlock A	
	lock A
	write A
write B	
unlock B	
	lock B
	write B
	write A
	unlock A
	write B
	unlock B

Schedule S_1

T_1	T_2
	lock A
	write A
	lock B
	write B
	write A
	unlock A
lock A	
write A	
	write B
	unlock B
lock B	
write B	
unlock B	
unlock A	

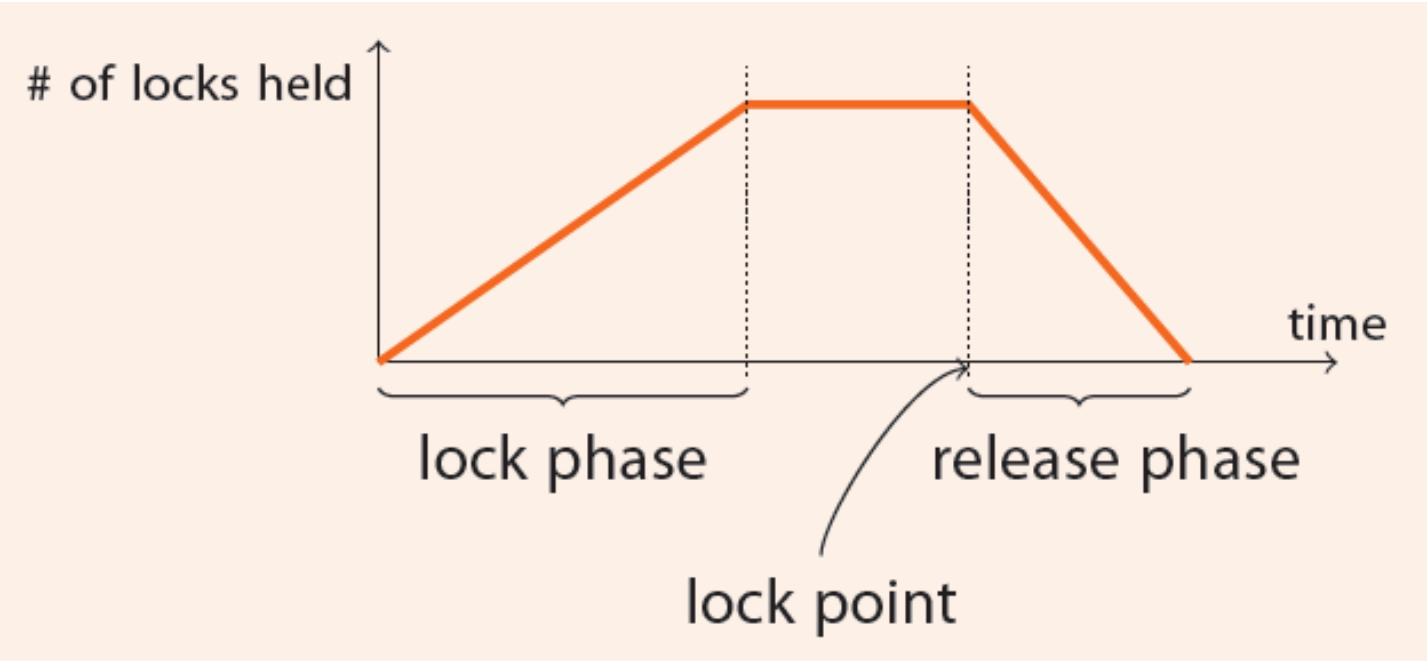
- Note: Both schedules $S_{1,2}$ are serializable. **Are we done yet?**

Locking and Serializability

Transaction 1	Transaction 2	DB state
lock (*acct*) ;		1200
read (*acct*) ;		
unlock (*acct*) ;		
	lock (*acct*) ;	
	read (*acct*) ;	
	unlock (*acct*) ;	
lock (*acct*) ;		
write (*acct*) ;		1100
unlock (*acct*) ;		
	lock (*acct*) ;	
	write (*acct*) ;	1000
	unlock (*acct*) ;	

Two-Phase Locking (2PL)

- Once a transaction has released a lock (i.e., performed the first unlock), it must not acquire any new lock.
- Two-phase locking is **the** concurrency control protocol used in traditional database systems.



Example: ATM Transaction Without 2PL

Transaction 1	Transaction 2	DB state
lock (*acct*) ;		1200
read (*acct*) ;		
unlock (*acct*) ;		
	lock (*acct*) ;	
	read (*acct*) ;	
	unlock (*acct*) ;	
lock (*acct*) ;		
write (*acct*) ;		1100
unlock (*acct*) ;		
	lock (*acct*) ;	
	write (*acct*) ;	1000
	unlock (*acct*) ;	

A 2PL-Compliant ATM Transaction

- To comply with the two-phase locking protocol, the ATM transaction must not acquire any new locks after a first lock has been released

```
lock (acct) ;                                  } lock phase
bal ← read_bal (acct) ;
bal ← bal − 100 € ;
write_bal (acct, bal) ;
unlock (acct) ;                                } unlock phase
```

Resulting Schedule

Transaction 1	Transaction 2	DB state
lock (*acct*) ;		1200
read (*acct*) ;		
	lock (*acct*) ;	
write (*acct*) ;	Transaction blocked	1100
unlock (*acct*) ;		
	lock (*acct*) ;	
	read (*acct*) ;	
	write (*acct*) ;	900
	unlock (*acct*) ;	

Why Does 2PL Work?

- 2PL Ensures Conflict-Serializability: all operations of each transaction can be moved to the place of the first unlock

- Proof: induction on the number of transactions
  - base case n = 1 is trivial: any schedule of one transaction is serializable

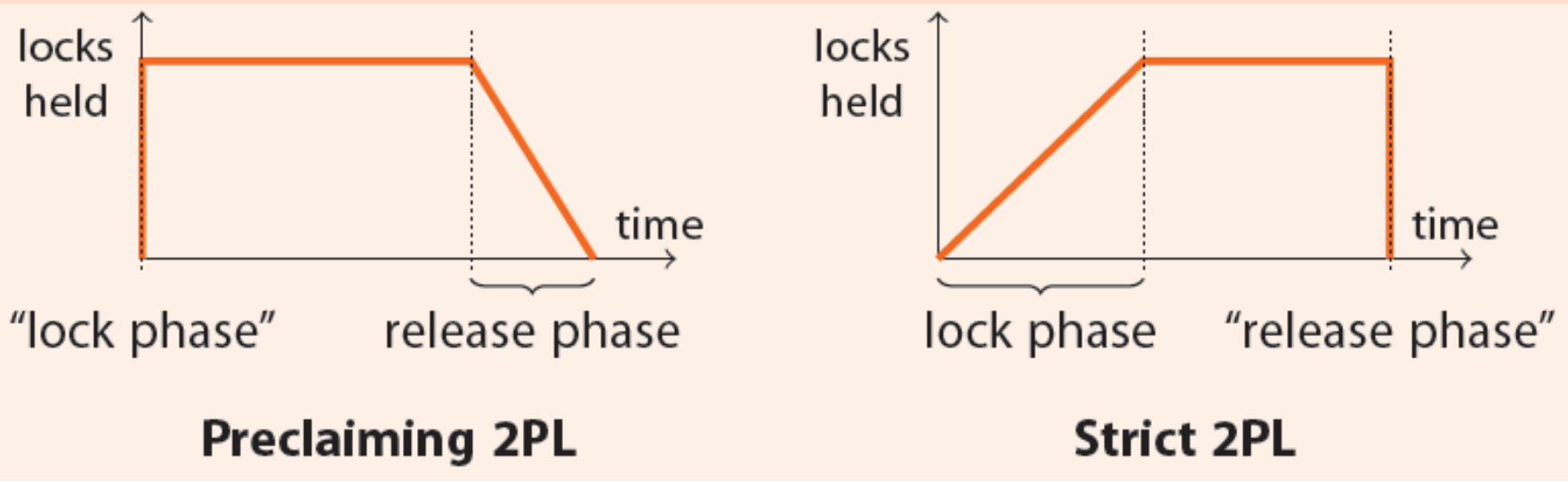
Why Does 2PL Work?

Induction step: take any 2PL schedule S of n transactions

- Find the first unlock operation $u_i(X)$ (of transaction T_i)
- Move each operation $a_i(Y)$ of T_i to the front:
  - Assume that S contains an operation $b_j(Y)$ that precedes $a_i(Y)$ and conflicts with it.
  - Then, T_j must unlock Y before $a_i(Y)$, but after $u_i(X)$ since $u_i(X)$ is the first unlock operation in S.
  - For $a_i(Y)$ to take place, T_i must acquire a lock on Y after $u_i(X)$.
  - Contradiction: S was not 2PL!
- Reduce the rest of the schedule by inductive assumption

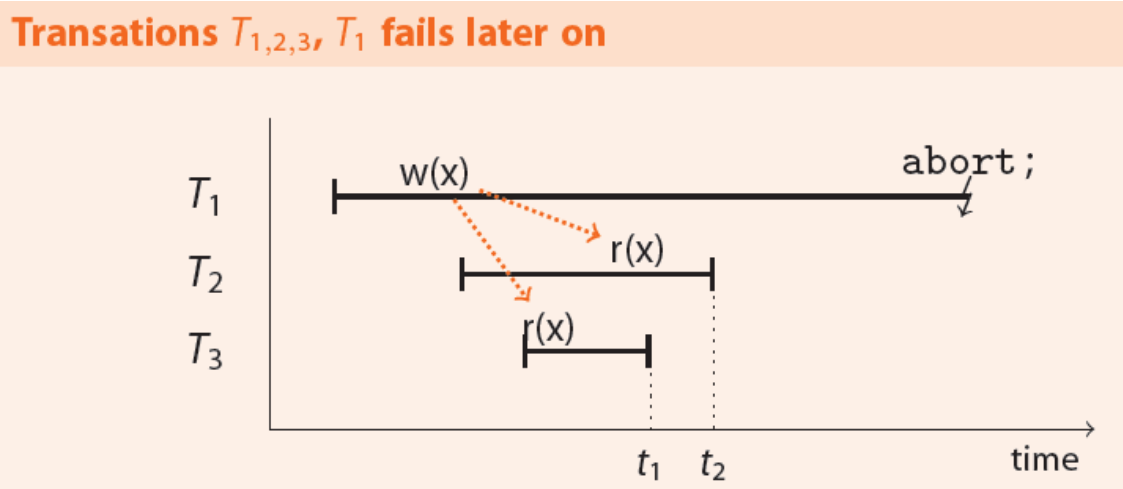
Variants of Two-Phase Locking

- 2PL does not prescribe exactly when locks should be acquired and released.
- Two possible variants: pre-claiming and strict 2PL



- Both variants only allow conflict-serializable schedules.

Cascading Rollbacks



- When transaction T_1 aborts, transactions T_2 and T_3 have already read data written by T_1 (dirty read)
- T_2 and T_3 need to be rolled back, too!
- T_2 and T_3 cannot commit until the fate of T_1 is known.
- Strict 2PL avoids cascading rollbacks:
  - all write locks released at the end

  ⇒ no transaction can read uncommitted data

Deadlocks

- 2PL can result in deadlock.
- Example:
  - Proper schedule with locking
  - Both transactions wait for each other indefinitely

Transaction 1	Transaction 2
`lock (A);`	
⋮	`lock (B);`
do something	⋮
⋮	do something
`lock (B);`	⋮
[wait for T_2 to release lock]	`lock (A);`
	[wait for T_1 to release lock]

Deadlock Detection

1. The system maintains a **waits-for graph**, where an edge from T_1 to T_2 indicates that T_1 is blocked by a lock held by T_2.
2. Periodically, the system tests for **cycles** in the graph.
3. If a cycle is detected, the deadlock is **resolved** by **aborting** one or more transactions.
4. Selecting the **victim** is a challenge:
   - Aborting **young** transactions may lead to **starvation**: the same transaction may be cancelled again and again.
   - Aborting an **old** transaction may cause a lot of computational investment to be thrown away (the **undo** costs may be high).

Timeout-Based Deadlock Detection

- Let a transaction T block on a lock request only until a **timeout** occurs (counter expires).
- On expiration, assume that a deadlock has occurred and abort T.

DB2 Timeout-based deadlock detection

```
db2 => GET DATABASE CONFIGURATION;
    :
Interval for checking deadlock (ms)      (DLCHKTIME) = 10000
Lock timeout (sec)                     (LOCKTIMEOUT) = 30
    :
```

Deadlock Prevention: Wait-Die

- Alternative: prevent deadlocks in the first place

1. Assign priorities to transactions based on timestamps.
2. When T_i tries to acquire a lock that T_j holds:
   - if T_i is older, T_i waits for T_j;
   - otherwise, T_i aborts.

- Known as the **Wait-Die** strategy
- All edges in the waits-for graph are $T_{older} \rightarrow T_{younger}$, so the graph is always acyclic!
- **Non-preemptive**: a transaction never gives up a lock

Deadlock Prevention: Wound-Wait

- Alternative: prevent deadlocks in the first place

1. Assign priorities to transactions based on timestamps.
2. When T_i tries to acquire a lock that T_j holds:
   - if T_i is older, abort T_j;
   - otherwise, T_i waits for T_j.

- Known as the **Wound-Wait** strategy
- All edges in the waits-for graph are $T_{younger} \rightarrow T_{older}$, so the graph is always acyclic!
- **Preemptive**: a younger transaction is killed

Deadlock Prevention and Starvation

- Either way, only younger transactions are aborted and usually restarted.
- **Problem:** a transaction can be restarted indefinitely by a long-running transaction
- **Solution:**
  - When restarting a transaction, reuse the original timestamp.
  - Old transactions finish eventually, which gives the newcomers a chance to finish.

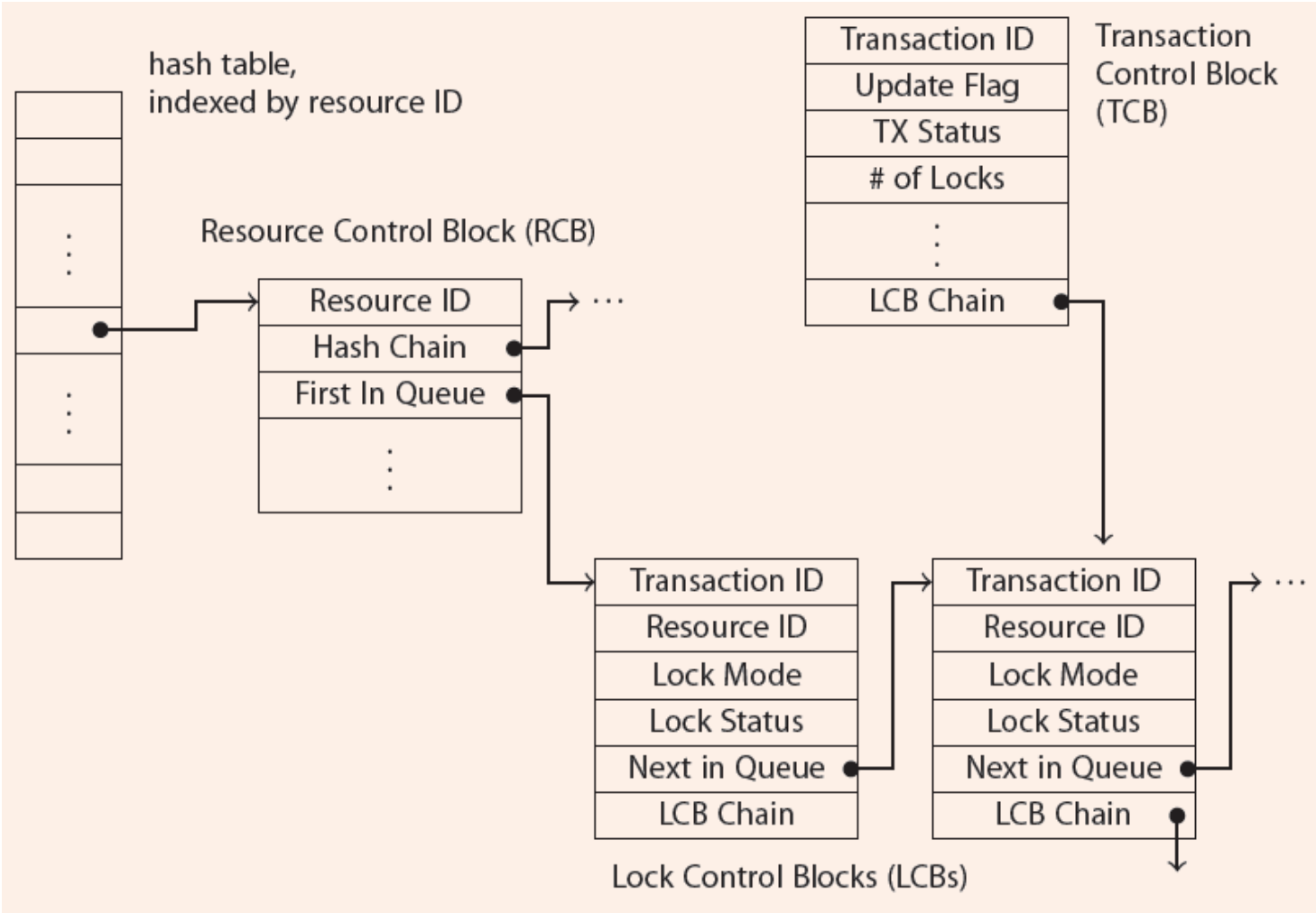
Implementing a Lock Manager

The Lock Manager should do three tasks very efficiently:

1. Check which locks are currently held for a given **resource** (invoked on lock(A) to decide whether the lock request on A can be granted).
2. When unlock(A) is called, **blocked transactions** that **requested** locks on A must be identified and now granted the lock.
3. When a transaction **terminates**, all held locks must be released.

What is a good data structure to accommodate these needs?

Lock Manager: Bookkeeping

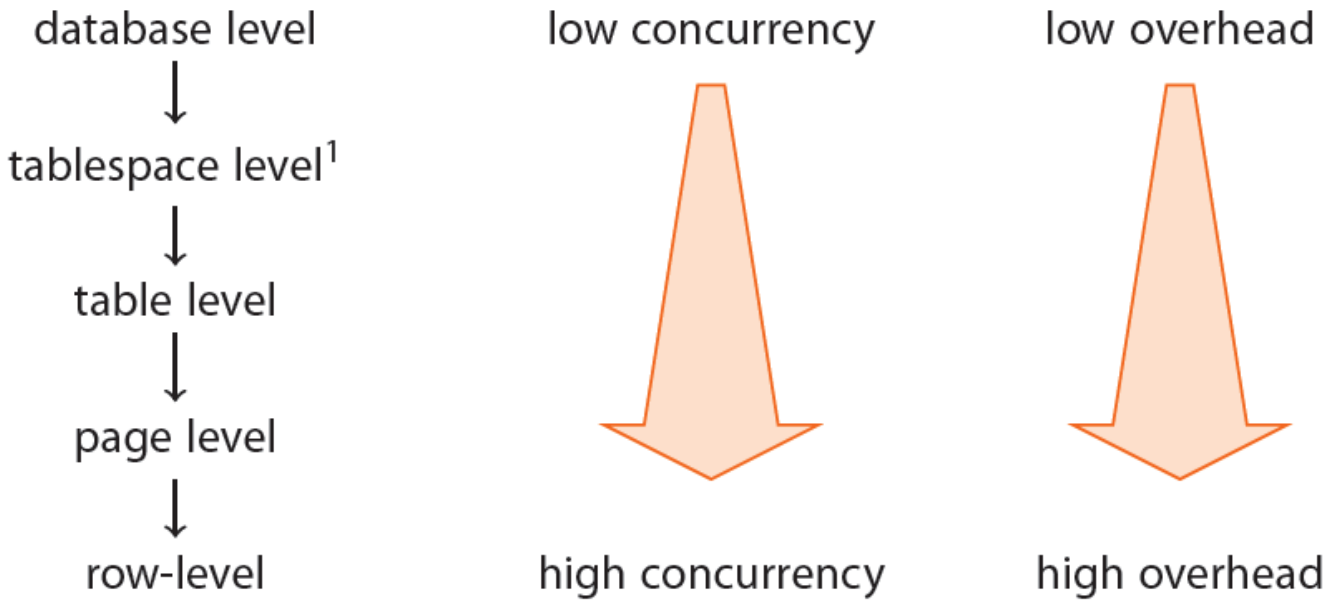


Implementing Lock Manager Tasks

- The locks held for a given **resource** can be found using a **hash lookup**:
  - Linked list of Lock Control Blocks via ‘First/Next In Queue’
  - The list contains **all** lock requests, granted or not.
  - The transaction(s) at the **head** of the list are the ones that currently hold a lock on the resource.
- When a lock is **released** (i.e., its LCB is removed from the list), the next transactions in the list are considered for granting the lock.
- All locks held by a single **transaction** can be identified via the linked list ‘LCB Chain’ (and easily released upon transaction termination).

Granularity of Locking

- The granularity of locking is a trade-off:



- Idea: multi-granularity locking

[1]DB2 tablespace = collection of tables that share a physical storage location.

Multi-Granularity Locking

- Decide the granularity of locks held for each transaction (depending on the characteristics of the transaction)
- For example, acquire a **row lock** for query 1 (top) and a **table lock** for query 2 (bottom)
- Locking is **performance-critical**. Query 2 does not want to search for row-level conflicts when trying to lock the table.
- How do transactions know about each others' locks?

Row-selecting query (C_CUSTKEY is key)

```
SELECT *
FROM   CUSTOMERS
WHERE  C_CUSTKEY = 42
```

Table scan query

```
SELECT *
FROM   CUSTOMERS
```

Intention Locks

Databases use an additional type of locks: intention locks.

- Lock mode intention share: IS
- Lock mode intention exclusive: IX

Conflict matrix shown on the right

	S	X	IS	IX
S		×		×
X	×	×	×	×
IS		×		
IX	×	×		

- A lock I*a* on a coarser level of granularity means that there is some *a* lock on a lower level.

Intention Locks

Multi-granularity locking protocol:

- Before a granule g can be locked in *p* mode (S or X), the transaction must obtain an I*p* lock on **all** coarser granularities that contain g.
  - Based on the containment hierarchy of files, tablespaces, tables, pages, records
- If all intention locks could be granted, the transaction can lock granule g in the desired mode.
- Locks must be released from smaller to coarser granularities

Multi-Granularity Locking: Example

Query 1 obtains:

- IS lock on the database
  (and table space if applicable)
- IS lock on CUSTOMERS
- S lock on the **row** with C_CUSTKEY = 42.

Row-selecting query (C_CUSTKEY is key)

```
SELECT *
FROM   CUSTOMERS
WHERE  C_CUSTKEY = 42
```

Query Q2 obtains:

- IS lock on the database
  (and tables space if applicable)
- S lock on table CUSTOMERS

Table scan query

```
SELECT *
FROM   CUSTOMERS
```

Multi-Granularity Locking: Conflicts

- The update acquires:
  - IX lock on **table** CUSTOMER (and all coarser granules)
  - X lock on the **row** holding customer 17.
- The update is **compatible** with Query 1
  - There is no conflict between IX and IS on the table level.
- The update is **incompatible** with Query 2
  - The S table lock by Query 2 is in conflict with IX lock request.

Update request

```
UPDATE CUSTOMERS
SET NAME = 'Seven␣Teen'
WHERE C_CUSTKEY = 17
```

Lock Escalation: an Aside

Lock escalation dynamically asks for coarser-grained locks when too many low-level locks acquired.

- Example: T_1 uses an index to read only part of R
  - Gets IS lock on R, and repeatedly gets an S lock on relevant tuples of R.
  - Alternative: Use lock escalation to upgrade from IS to S lock on R.

Consistency Guarantees and SQL 92

- Sometimes, some degree of inconsistency may be acceptable for specific applications:
- Occasional ‘off-by-one errors’ will not considerably affect the outcome of an aggregate over a huge table.
  - Accept inconsistent read anomaly and be awarded with improved concurrency.
- To this end, SQL 92 specifies different **isolation levels.**

DB2 SQL 92 isolation level control

```
SET ISOLATION SERIALIZABLE | DIRTY READ | READ UNCOMMITED | ···;
```

- Relaxed consistency guarantees should lead to increased throughput.

SQL 92 Isolation Levels

READ UNCOMMITTED (lowest isolation level, highest concurrency)

- Only write locks are acquired (according to strict 2PL). Any **row read may be concurrently changed** by other transactions.

READ COMMITTED

- Read locks are only held for as long as the application's cursor sits on a particular **current row**. Other rows may be changed by other transactions. Write locks acquired according to strict 2PL.

READ STABILITY (REPEATABLE READS)

- A transaction may read **phantom rows** if it executes an aggregation query twice; read/write locks kept until the end of TX, no range locks

SERIALIZABLE (highest isolation level, lowest concurrency)

- Strict 2PL locking. **No phantom rows** seen by transaction.

Phantom Row Problem

Transaction 1	Transaction 2	Effect
scan relation R;		T_1 locks all rows
	insert new row into R;	T_2 locks new row
	`commit;`	T_2's lock released
scan relation R;		reads **new** row, too!

- Both transactions properly followed the 2PL protocol, but T_1 observed an effect of T_2 ⇒ isolation violated!
- Cause of the problem: T_1 can only lock **existing** rows.
- Solution 1:
  - treat insertion into R as write on R (so request a write lock on R)
  - bad for concurrency
- Solution 2: describe locked rows **declaratively**
  - **Key range locking**, typically in B+ trees
  - **Predicate locking** (if predicate evaluates to true for row, lock it)

Key Range Locking

SELECT * FROM customers WHERE c_custkey = 42

- If there is a dense index on c_custkey, we can lock the index page for data entries with c_custkey = 42
  - Even if there are no records with c_custkey = 42, we lock the index page where such a data entry would be, if it existed!
- If there is no suitable index, we must lock all pages, and lock the file/table

- This prevents a concurrent insertion of a customer with c_custkey = 42

Predicate Locking

SELECT * FROM customers WHERE c_custkey = 42

- Describe a lock **declaratively** using a data structure describing a condition for the lock
  - E.g., a list of equations of the form R.A θ v, where R.A is a relation attribute, θ is <, =, >=, etc., and v is a value

- Predicate locks are
  - asserted during query evaluation, and
  - checked during insertion.

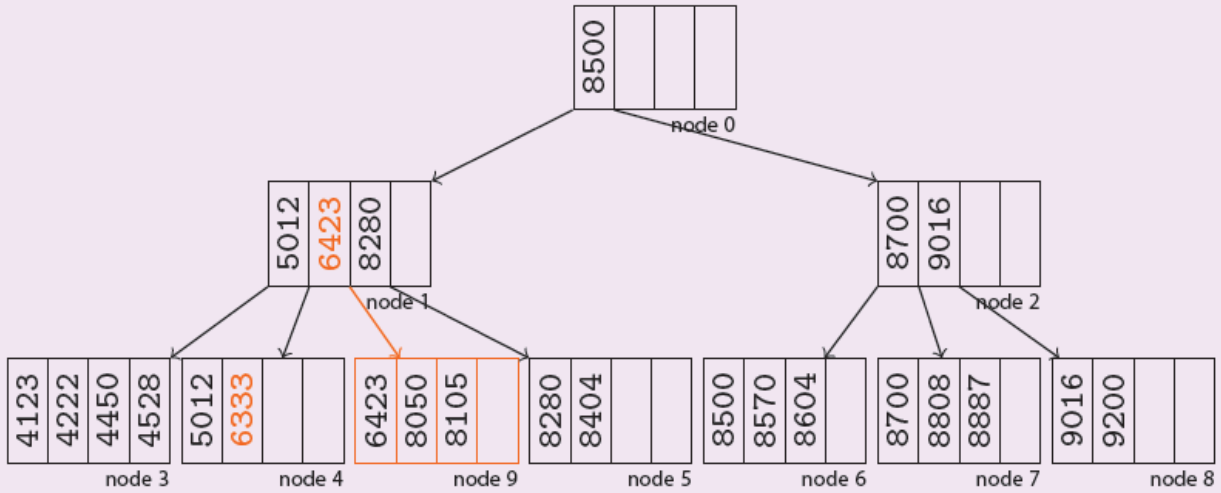
Isolation levels vs. Consistency Guarantees

isolation level	dirty read	non-repeat. rd	phantom rd
READ UNCOMMITTED	possible	possible	possible
READ COMMITTED	not possible	possible	possible
READ STABILITY	not possible	not possible	possible
SERIALIZABLE	not possible	not possible	not possible

- Non-repeatable read occurs a transaction reads a row twice, and its values differ between reads.
- Some DBMS implementations support more, less, or different levels of isolation.
- Many applications do not need serializability. Choosing a weaker yet acceptable level is part of database tuning.

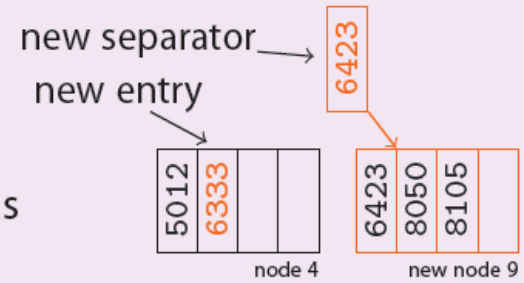
Concurrent Access to B+ Trees

Example (A search for key 8050 may be misguided)



Insert key 6333.

- Must **split** node 4.
- **New separator** goes into node 1 (only after lookup for key 8050 has been performed).



T_1 inserts 6333:

- old node 4 is split into nodes 4 and 9
- new separator 6423 inserted into node 1

T_2 searchers for 8050:

- reads node 1 before the separator is inserted

⇒ goes to node 4

- but node 4 has already been modified

⇒ T_2 misses 8050!

Locking and B+ Tree Indices

We **search** a B+ tree **top down:**

- in each node we decide which child to continue with.

To **update** a B+ tree:

- we first do a **search**,
- we then insert new data into the right **leaf**, and
- depending on the **fill levels** of nodes, we **split** tree nodes and propagate splits **bottom up**.

According to the 2PL protocol, we would have to

- obtain S/X locks when we walk down the tree, and
- **keep** all locks until we are finished.

This strategy would seriously reduce concurrency:

- all transactions would lock the tree root which effectively serializes all (write) transactions.

Tree-Locking Protocol

Each transaction must follow these rules:

- The first lock can be on any node.
- Each subsequent lock can be acquired if the transaction currently has a lock on the parent node.
- A node can be unlocked *at any time*.
  - Relaxes two-phase locking!
- A node that has been unlocked cannot be relocked (not even if the transaction still holds a lock on the parent).

Effectively:

- All transactions follow a top-down access pattern.
- No transaction can ‘bypass’ any other transaction along the same path, so conflicting transactions are thus serializable.
- The protocol is deadlock free.

Split Safety

- We may still need to keep as many write locks since nodes might be affected by node splits.
- However, if node n contains less than 2d entries, no split will propagate above n
  - Node n it is said to be (split) safe.
- We can use this to release write locks early:
  - If we encounter a safe node n during insertion, we can release locks on all of n's ancestors.

⇒ Locks near the root are held for a shorter time

Lock Coupling Protocol (Variant 1)

Readers

```
place S lock on root ;
current ← root ;
while current is not a leaf node do
    place S lock on appropriate son of current ;
    release S lock on current ;            /* lock coupling */
    current ← son of current ;
```

Writers

```
place X lock on root ;
current ← root ;
while current is not a leaf node do
    place X lock on appropriate son of current ;
    current ← son of current ;
    if current is safe then
        release all locks held on ancestors of current ;
```

Increasing Concurrency for Common Scenarios

- Even with lock coupling, inner tree nodes can often become locked, which reduces concurrency.

- Chances that an inner node is affected by an update is very small.
  - Back-of-the-envelope calculation:

    For d = 50, every 50th insert causes a split (2% chance).

- An insert transaction could thus optimistically assume that no leaf split is going to happen:
  - On inner nodes, acquire only read locks during tree navigation (plus a write lock on the affected leaf).
  - If assumption is wrong, nothing is broken yet: re-traverse the tree and obtain write locks.

Lock Coupling Protocol (Variant 2)

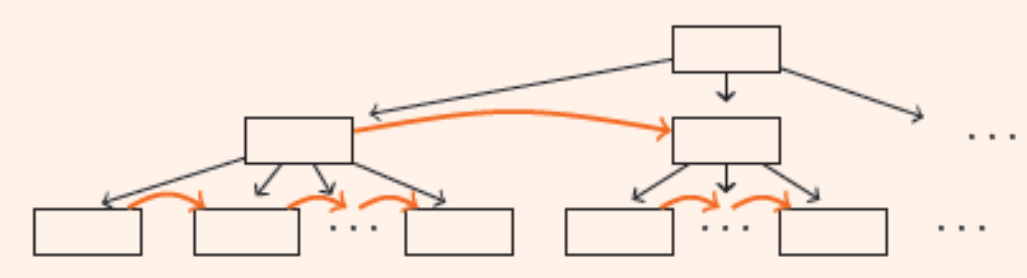
Modified protocol for writers (readers protocol unchanged):

Writers

```
place S lock on root ;
current ← root ;
while current is not a leaf node do
    son ← appropriate son of current ;
    if son is a leaf then
        place X lock on son ;
    else
        place S lock on son ;
    release lock on current ;
    current ← son ;
/* current is a leaf now                    */
if current is unsafe then
    release all locks and repeat with protocol Variant 1 ;
```

B+ Trees Without Read Locks

- Lehman and Yao (TODS 1981) proposed a protocol that does not need any read locks on B+ tree nodes.
- Requirement: a *next* pointer, pointing to the right sibling.

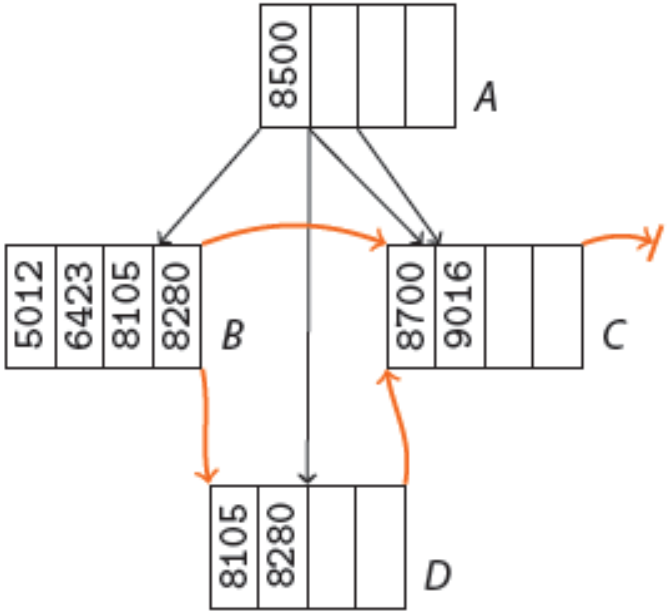


- Original B+ tree: linked list along the **leaf level** only (sequence set).
- Sibling pointers provide a **second path** to find each node.
- This way, read operations that have been misguided by concurrent splits can still find the node that they need.

Insertion With Inner Node Split

Insertion (of key 4222) hits full inner node *B*

lock & read page *B* ;
create new page *D* and lock it ;
populate page *D* ;
set `next` pointer $D \rightarrow C$;
write *D* ;
set `next` pointer $B \rightarrow D$;
adjust content of *B* ;
write *B* ;
lock & read *A* ;
adjust content of *A* ;
write *A* ;



All index entries (not pages) remain reachable at all times.

B+ Tree Access By Readers

- The next pointers give readers the chance to ‘find’ entries even in the middle of a split, when some entries (e.g., 8105, 8280) have already moved to the new page.
- During tree_search(k), move from n to the next sibling if
  - no appropriate entry can be found in n (high key in n is less than k, so the tree structure may be temporarily inconsistent), and
  - next is not a null pointer.
- A search for key 8200 exceeds the high key value of node B (which is 8105), so we traverse the sibling link from node B to D.
- Write locks do not prevent readers from accessing a page!
  - This is because readers do not acquire read locks.
- PostgreSQL uses this protocol for B+ tree access.

Optimistic Concurrency Control

- Our approach has thus far been **pessimistic**
  - Assume that transactions **will conflict** and thus protect database elements by locks and lock protocols.
- The converse = **optimistic concurrency control**
  - Hope for the best and let transactions freely proceed with their read/write operations.
  - Only just before updates are to be committed to the database, perform a check to see whether conflicts indeed did not happen.
- Rationale: Non-serializable conflicts are not that frequent. **Save the locking overhead** in the majority of cases and only invest effort if really required.

Optimistic Concurrency Control

1. **Read Phase.** Execute transaction, but do not write data back to disk immediately. Instead, collect updates in the transaction's **private workspace**.
2. **Validation Phase.** When a transaction wants to commit, test whether its execution was correct (only acceptable conflicts happened). If it is not, abort the transaction, clear its workspace, and restart it.
3. **Write Phase.** Transfer data from private workspace into database.

Phases 2 and 3 need to be performed in a non-interruptible critical section (thus also called the val-write phase).

Validating Transactions

Validation is implemented by looking at transaction T's

- read set RS (elements read by T) and
- write set WS (elements written by T).

Backward-oriented optimistic concurrency control (BOCC):

- Validation of T succeeds if, for each committed transaction U, either U committed before T started, or $RS(T) \cap WS(U) = \emptyset$.

Forward-oriented optimistic concurrency control (FOCC):

- Validation of T succeeds if, for each currently running transaction U, $WS(T) \cap RS(U) = \emptyset$.

Concurrency Control via Versioning

Example schedule: $r_1(X)$, $w_1(X)$, $r_2(X)$, $\overset{t}{\downarrow}$ $w_2(Y)$, $r_1(Y)$, $w_1(Z)$

- **not** serializable

- However, if T_1 can read the 'old' value of Y at time *t* (prior to the conflicting $w_2(Y)$ operation), we can create the **serializable** schedule

  $r_1(X)$, $w_1(X)$, $r_2(X)$, $\underline{r_1(Y)}$, $w_2(Y)$, $r_1(Y)$, $w_1(Z)$

  – the underlined read $\underline{r_1(Y)}$ refers to a read of an older value of Y.

- Downside: space overhead for version maintenance

Snapshot Isolation

- New concurrency level:
  - Used in Oracle, SQL Server, PostgreSQL, SAP HANA
  - Berenson, H., Bernstein, P., Gray, J., Melton, J., O'Neil, E., and O'Neil, P. A critique of ANSI SQL isolation levels. SIGMOD 1995
- General principles:
  - Each transaction reads a snapshot that the database was in at transaction start
    - No read locks needed
    - Readers and writers can proceed independently ⇒ promotes concurrency!
  - When two transactions attempt to write to an element, one transaction wins, and another is aborted
    - first updater wins
    - first committer wins

Write Skew in Snapshot Isolation

Constraint: ACC1 + ACC2 ≥ 100

T_1	ACC1	ACC2	T_2
	100	100	
BEGIN			BEGIN
Read(ACC1) → 100			
Read(ACC2) → 100			
Write(ACC1,0) → OK!	0		
			b_1 := Read(ACC1) → 100
			b_2 := Read(ACC2) → 100
		0	Write(ACC2,0) → OK!
COMMIT			COMMIT

ACC1 = 100 in the snapshot of T_2!

Both succeed because T1 and T2 write different elements!

SI is stronger than `REPEATABLE READ`, but not `SERIALIZABLE`.

Multiversion Concurrency Control (MVCC)

- Often used to implement snapshot isolation
  - But can be used to implement other levels too
- Idea:
  - Keep old element versions with timestamps
  - Update = delete old + create new version
- E.g., each row in PostgreSQL has two fields
  - xmin: the timestamp of the transaction that introduced the row
  - xmax: the timestamp of the transaction that deleted the row (or 0 if the row has not been deleted)
  - Versions treated as distinct elements (e.g., added to indexes)

Implementing Snapshots

- Each transaction T records…
  - xid: the ID of the transaction
    - Incremented when transaction starts
  - Set R of IDs of running transaction when T started (not containing xid)
- Row is visible to T if…
  - xmin ≤ xid and xmin ∉ R, and
  - xmax = 0, or xmax ≥ xid and xmax ∉ R
- Another variant: get old versions from undo log

Implementing “First Updater Wins”

- Transaction T must lock a row to update it
  - after inserting a new row, or
  - to write xmax into an existing row
- If the row is locked by some T’, suspend T
  - If T’ commits, abort T
  - If T’ aborts, resume T

⇒ Conflicts detected as updates happen

⇒ Deadlock possible!

Implementing “First Committer Wins”

- Updates are kept in a private workspace and transferred to the database in an atomic commit phase
  - commits cannot progress in parallel

- If an item was updated by a concurrent transaction, commit is aborted

⇒ Conflict checking delayed until commit time

Serializing Snapshot Isolation

- Variant 1: use two-phase locking for updates

- Variant 2: maintain a **read timestamp** RT(X) for each version X
  - When T creates X, set RT(X) = TS(T)
  - When T reads X and TS(T) > RT(X), set RT(X) = TS(T)
  - T can create a new version of X only if RT(X) ≤ TS(T)

Serializing Writes using Read Timestamps

T_1	T_2	T_3	
Create X_1			$WT(X_1) := RT(X_1) := 1$
Create Y_1			$WT(Y_1) := RT(Y_1) := 1$
		Create Y_3	$WT(Y_3) := RT(Y_3) := 3$
	Create Y_2		$WT(Y_2) := RT(Y_2) := 2$
		Read X_1	$RT(X_1) := 3$
	Create X_2		

Not a problem:

- T_2 sees Y_1, and T_3 has not read Y_1

Problem:

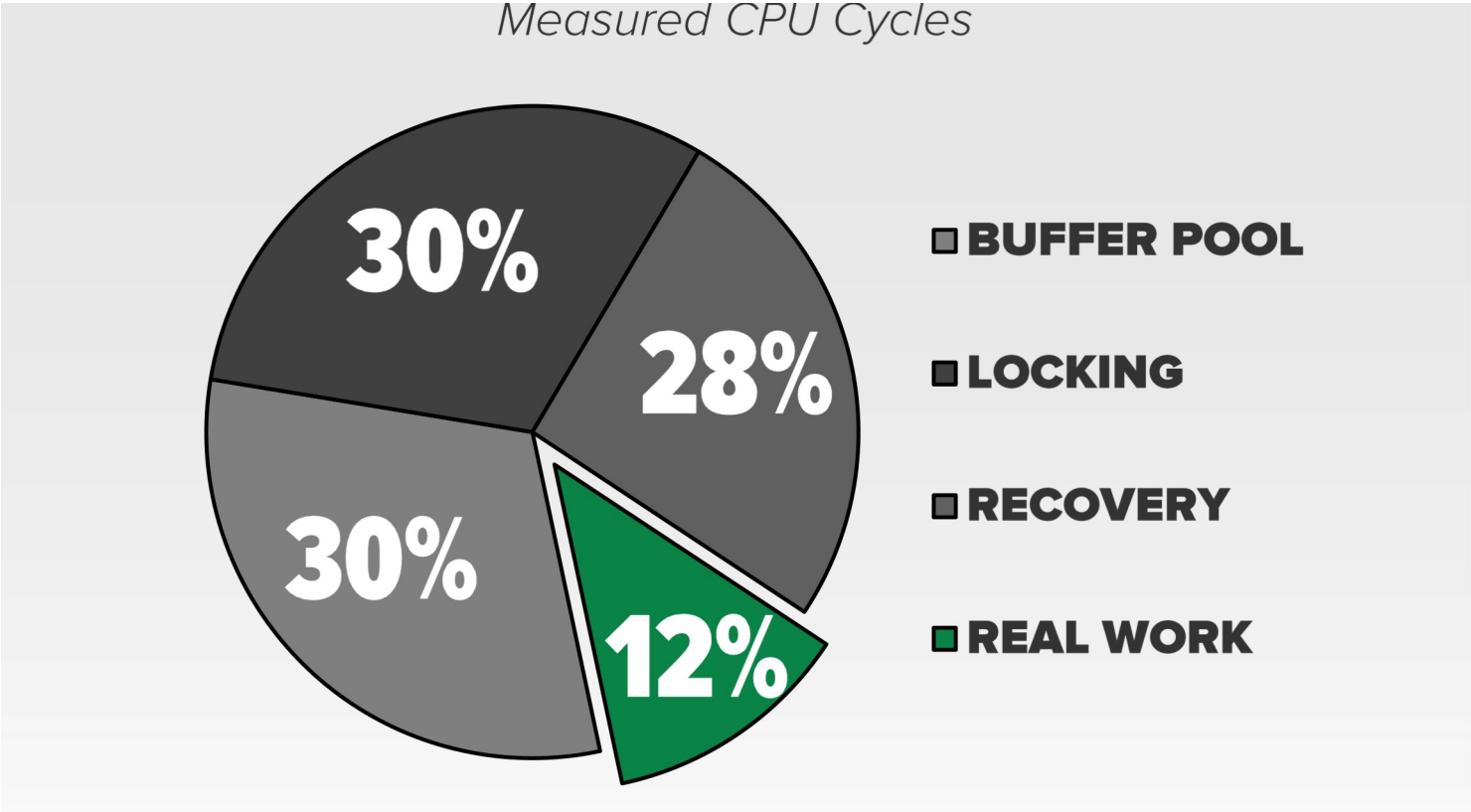
- T_3 read X_1, so writing X_2 would invalidate T_3's read

$\Rightarrow$ T_2 is aborted

Serializing Snapshot Isolation

- Solution has the phantom row problem:
  - Start T_1, $TS(T_1) = 1$
  - Start T_2, $TS(T_2) = 2$
  - SELECT * FROM R in T_2 returns nothing
  - T_1 inserts a row X into R
    - $TS(X) = RT(X) = TS(T_1) = 1$
    - Allowed because T_2 has not read X
  - T_2 should have read X in the SELECT query!
    - **Not serializable**!
- Solution requires predicate locks!
- Cahill, M.J., Röhm. U., Fekete, A.D. Serializable Isolation for Snapshot Databases. ACM TODS, 2009

Traditional DBMS: Time for a Rewrite?



OLTP through the Looking Glass and What We Found There.
SIGMOD 2008. MIT&Brown folks. (Picture from Pavlo's talks)

Examples of State-of-the-Art Concurrency Control

- Application-level specification of constraints that reduce the degree of coordination needed by concurrent threads
  - *Coordination-Avoiding Database Systems.* AMPlab folks. March 2014. http://arxiv.org/pdf/1402.2237.pdf
- Transaction Repair @LogicBlox Inc.
  - Multi-version Optimistic Concurrency Control: Transactions run in parallel on different branches of the databases
  - Repair: If conflicts detected then update later conflicting transactions using *incremental* maintenance
  - *Transaction Repair: Full Serializability Without Locks.* LogicBlox folks. March 22, 2014. http://arxiv.org/pdf/1403.5645v1.pdf
- Use models to predict transaction behavior *in advance*
  - *On Predicting Modeling for Optimizing Transaction Execution in Parallel OLTP Systems*. MIT&Brown folks. VLDB 2011.